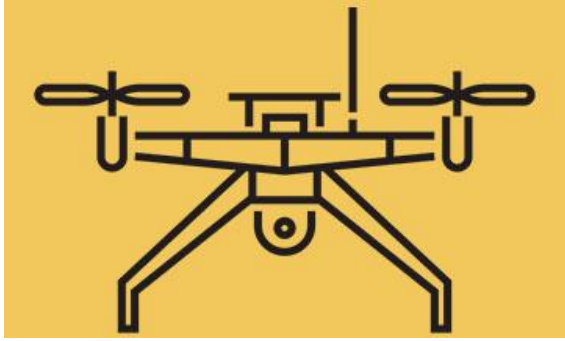
Feedforward control and differential flatness

Jerome Jouffroy, Professor
jerome@sdu.dk

SDU

Today's lecture

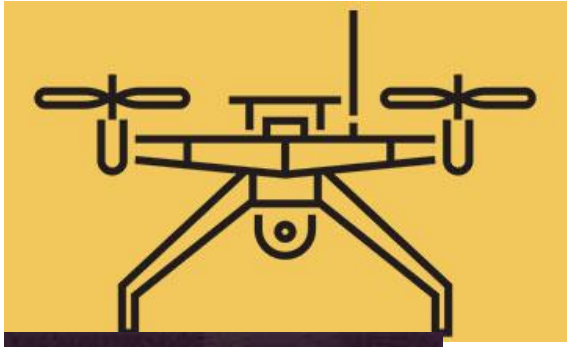


- On aggressive maneuvers and feedforward control
- Flatness intuition on a simple example
- Differential Flatness theory
- The two-wheel robot is flat
- Quadcopters are flat
- Application examples in video

"Aggressive" maneuvers

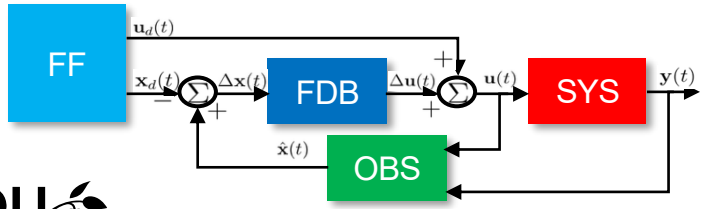
What is the common point between

and



?

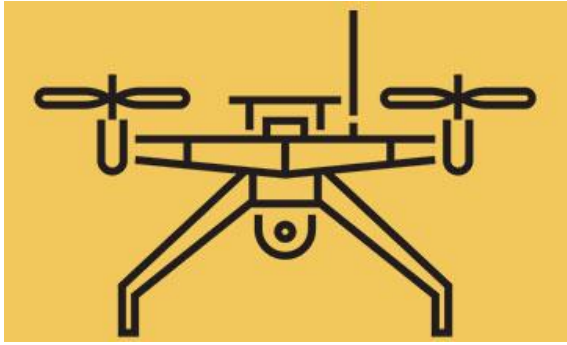
Feedforward control: compute a pre-planned control input that will make the system follow the desired behavior



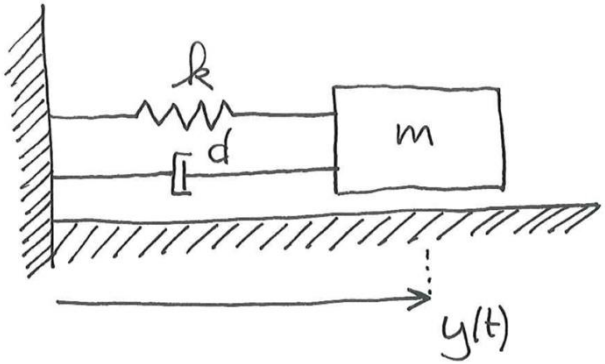
- Put less strain on the feedback controller
- Computationally more efficient

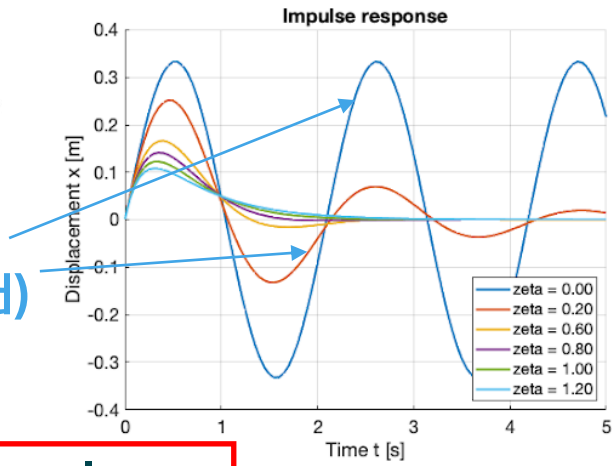
SDU

Looking at Feedforward on a simple example (1/2)



Consider a simple Mass-Spring-Damper system:





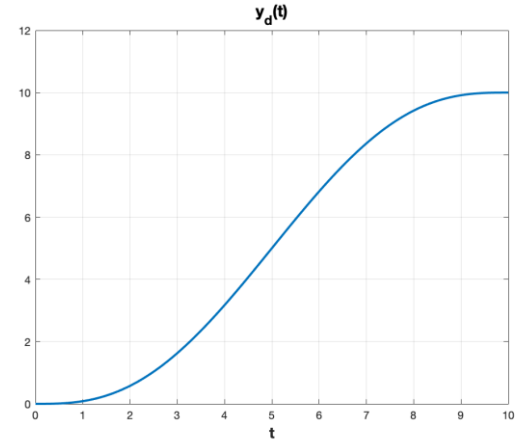
Mathematical model:

$$m\ddot{y}(t) + d\dot{y}(t) + ky(t) = u(t)$$ (assume MSD badly damped)

<u>Central question in Feedforward control</u>:

Starting from a position at rest, can I compute a control input to steer this position to a desired one in a specified time T?

We would like the system position to follow a pre-defined trajectory $y_d(t)$ over [0,T]

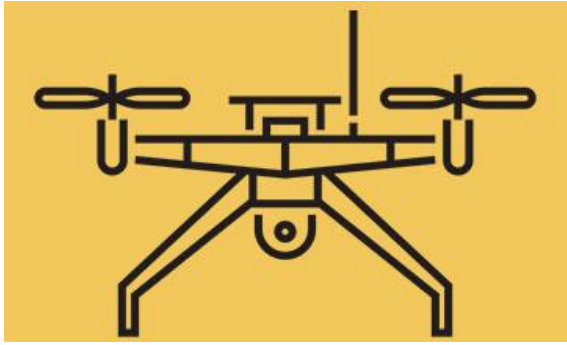


What is the corresponding desired control input $u(t)=u_d(t)$?

(only feedforward, no feedback!)

SDU

Looking at Feedforward on a simple example (2/2)



In the previous discussion, notice that we focus on/<u>choose</u> a specific variable (the position $y(t)=x_1(t)$), not on the whole state of the system.

Indeed, consider the SS-rep of MSD system

$$m\ddot{y}(t) + d\dot{y}(t) + ky(t) = u(t)$$

$$\begin{bmatrix} \dot{x}_1(t) \\ \dot{x}_2(t) \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -k/m & -d/m \end{bmatrix} \begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix} + \begin{bmatrix} 0 \\ 1/m \end{bmatrix} u(t)$$

with output equation $y(t) = x_1(t)$

This variable is quite central, <u>let us call it "flat" variable/output</u>:

$$\boxed{F(t) := x_1(t) = y(t)}$$

which gives $\dot{F}(t) = \dot{x}_1(t) = x_2(t)$ (state x is function of F and derivatives)

$$\ddot{F}(t) = \dot{x}_2(t)$$

Feedforward controller!

so that we ultimately have

$$\boxed{u(t) = kF(t) + d\dot{F}(t) + m\ddot{F}(t)}$$

(u is function of F and derivatives)

why is that interesting?

because if we want y(t) to follow $y_d(t)$ then $F(t) = y(t) = y_d(t)$

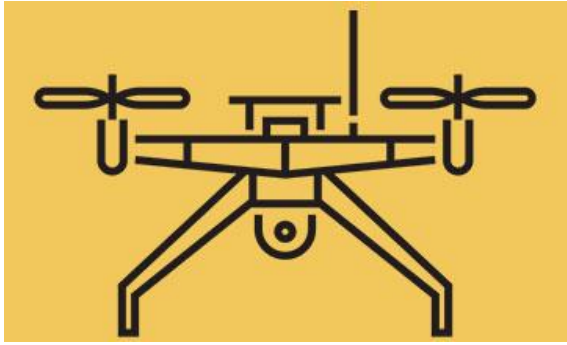
Assume now that $F(t)$ is a polynomial. Then $\dot{F}(t)$ is also a polynomial.

Then $\ddot{F}(t)$ is also a polynomial.

SDU

So that u(t) is a sum of polynomials! → We have computed u(t)!

Differential Flatness for feedforward control (in a nutshell)



Start with the mathematical model of your system (a differential equation of a car, boat, crane, etc.)

Choose/pick a flat variable/output *F(t)*

Express x(t) as a function F(t) and derivatives $\mathbf{x} = \mu_{\mathbf{x}}(F, \dot{F}, \ddot{F}, \cdots, F^{(q)})$ (1)

Express u(t) as a function F(t) and derivatives $u = \mu_u(F, \dot{F}, \ddot{F}, \cdots, F^{(q+1)})$ (2)

Use $\mathbf{x}_0$ and $\mathbf{x}_T$ to obtain $F(0), \dot{F}(0), \cdots$ and $F(T), \dot{F}(T), \cdots$

Plan desired trajectories $F_d(t)$ on [0,T] between $F(0), \dot{F}(0), \cdots$ and $F(T), \dot{F}(T), \cdots$

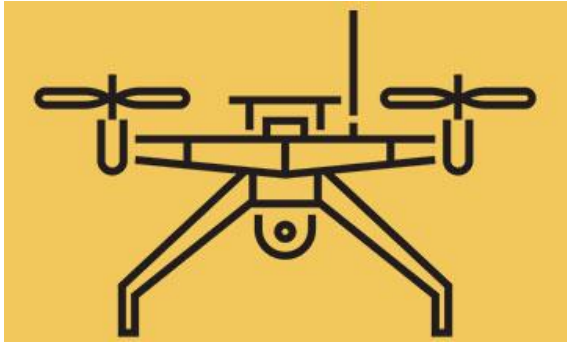
Finally, use (2) to get the feedforward controller

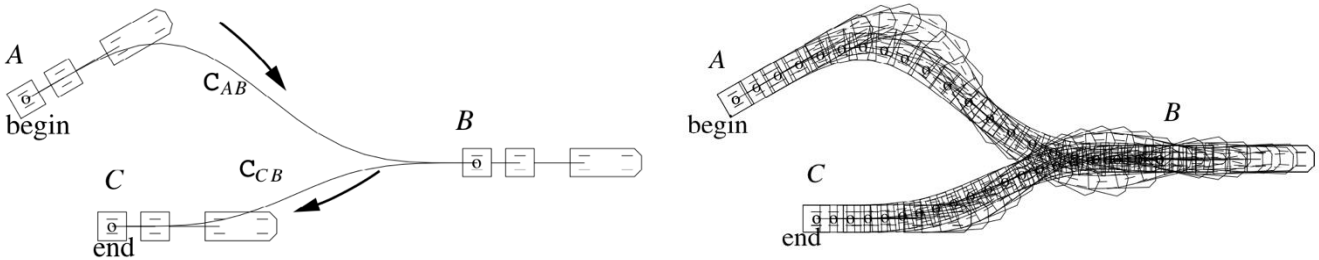
$$\boxed{u_d(t) = \mu_u(F_d(t), \dot{F}_d(t), \ddot{F}_d(t), \cdots, F_d^{(q+1)}(t))}$$

SDU

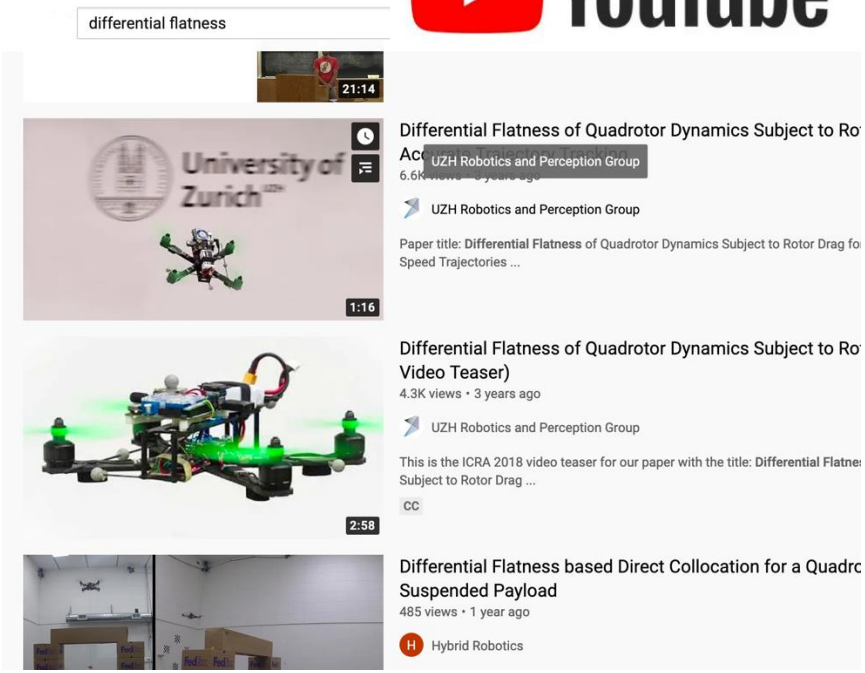
Differential Flatness theory

Where does it come from?

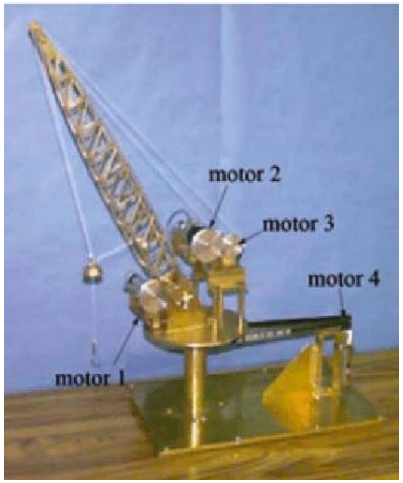




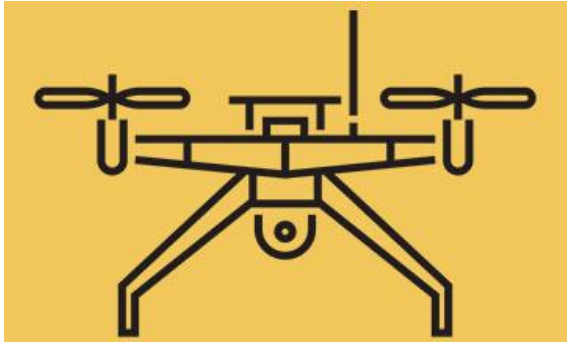
Nonholonomic car with trailers



Proposed by Michel Fliess and co-authors in 1995



SDU



Choosing the flat variable/output

As we have seen, differential flatness starts with a <u>choice</u>: the choice of the flat variable/output

Let us see on an example that making the right choice can be important:

Consider the linear system

$$\dot{\mathbf{x}} = \begin{bmatrix} 0 & 2 \\ 1 & -3 \end{bmatrix} \mathbf{x} + \begin{bmatrix} 1 \\ 0 \end{bmatrix} u$$

and <u>choose the flat variable</u> $F = x_1$

Then $\begin{cases} \dot{x}_1 = 2x_2 + u \\ \dot{x}_2 = x_1 - 3x_2 \end{cases} \Rightarrow \begin{cases} \dot{F} = 2x_2 + u \\ \dot{x}_2 = F - 3x_2 \end{cases}$

Isolate x_2? Not easy: u is in the way...

<u>Choose, instead, the flat variable</u> $F = x_2$

Then we have $\begin{cases} \dot{x}_1 = 2F + u \\ \dot{F} = x_1 - 3F \end{cases}$

Isolate x_1? Peace of cake! $x_1 = 3F + \dot{F}$

so that we get $\boxed{\begin{cases} x_1 = 3F + \dot{F} \\ x_2 = F \end{cases}}$

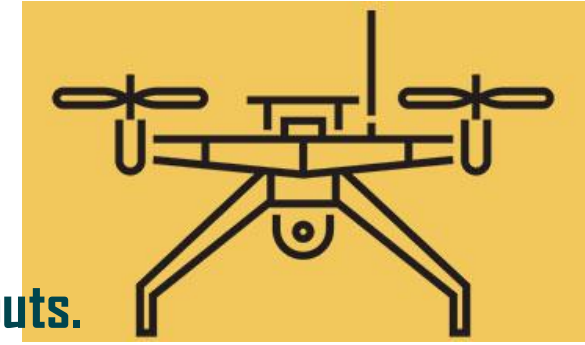
What about u? $u = -2F + \dot{x}_1$

$\Rightarrow \boxed{u = -2F + 3\dot{F} + \ddot{F}}$ $u = \mu_u(F, \dot{F}, \ddot{F}, \cdots, F^{(q+1)})$ $\mathbf{x} = \mu_{\mathbf{x}}(F, \dot{F}, \ddot{F}, \cdots, F^{(q)})$

SDU

Therefore the choice of the flat variable is important

The nonholonomic robot example (1/2)



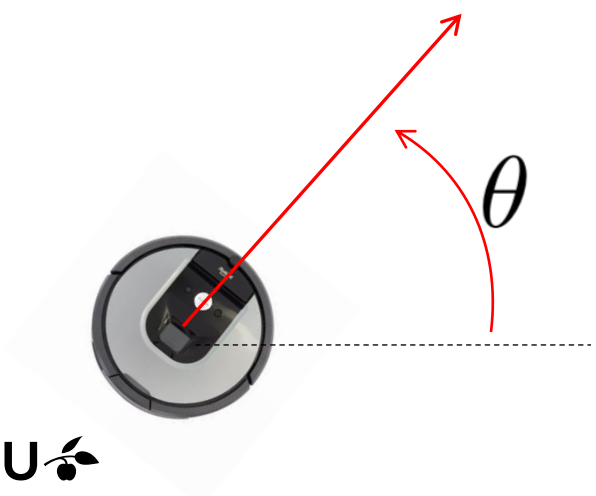
Differential flatness also applies to systems with several inputs.

In this case: the number of flat variables = the number of inputs!

F is now a vector of same dimension as the control input vector

$$\mathbf{u} = \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_p \end{bmatrix} \Rightarrow \mathbf{F} = \begin{bmatrix} F_1 \\ F_2 \\ \vdots \\ F_p \end{bmatrix}$$

Let us see an example:

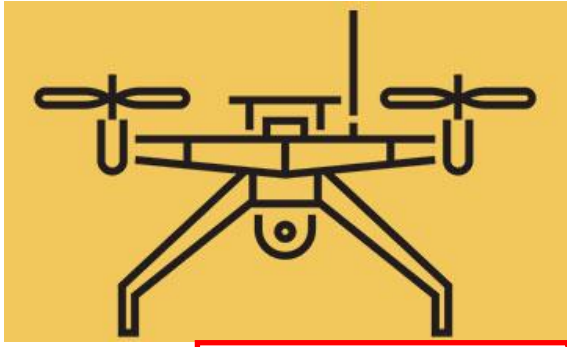


$$\begin{cases} \dot{x}_1 = u_1 \cos\theta \\ \dot{x}_2 = u_1 \sin\theta \\ \dot{\theta} = u_2 \end{cases}$$

(nonholonomic 2-wheel robot: Nonlinear system, dim(x)=3, dim(u)=2)

Feedforward control?

The nonholonomic robot example (2/2)



We have two inputs. Let us choose 2 flat variables:

$$F_1 = x_1 \quad \text{and} \quad F_2 = x_2$$

$$\begin{cases} \dot{x}_1 = u_1 \cos\theta & (1) \\ \dot{x}_2 = u_1 \sin\theta & (2) \\ \dot{\theta} = u_2 & (3) \end{cases}$$

Divide (2) by (1) (after replacement of x's by F's) :

$$\frac{\dot{F}_2}{\dot{F}_1} = \tan\theta \Longrightarrow \theta = \arctan\frac{\dot{F}_2}{\dot{F}_1}$$

$$\mathbf{x} = \begin{bmatrix} F_1 \\ F_2 \\ \arctan\left(\dot{F}_2/\dot{F}_1\right) \end{bmatrix}$$

We have $\mathbf{x} = \mu_{\mathbf{x}}(F, \dot{F}, \ddot{F}, \cdots, F^{(q)})$

Add square of (1) to square of (2):

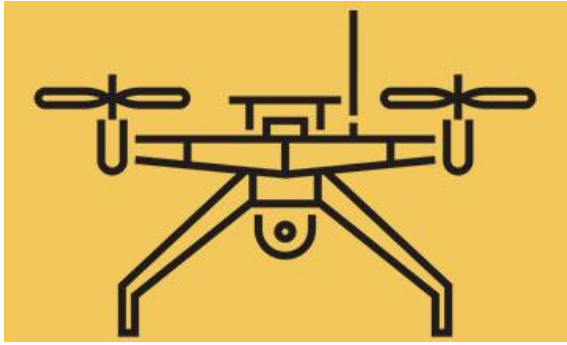
$$\dot{F}_1^2 + \dot{F}_2^2 = u_1^2(\cos^2\theta + \sin^2\theta) \Rightarrow \dot{F}_1^2 + \dot{F}_2^2 = u_1^2 \Longrightarrow u_1 = \pm\sqrt{\dot{F}_1^2 + \dot{F}_2^2}$$

$$\dot{\theta} = u_2 \Longrightarrow u_2 = \frac{d}{dt}\left(\arctan\left(\frac{\dot{F}_2}{\dot{F}_1}\right)\right) \Longrightarrow u_2 = \frac{\ddot{F}_2\dot{F}_1 - \ddot{F}_1\dot{F}_2}{\dot{F}_1^2 + \dot{F}_2^2}$$

We have the feedforward controller expression $u = \mu_u(F, \dot{F}, \ddot{F}, \cdots, F^{(q+1)})$

SDU

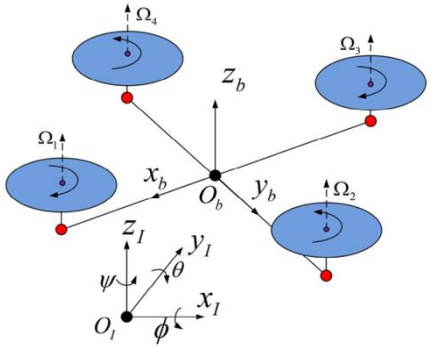
Drones are differentially flat! (1/3)



What about drones? Are they "flat"?

$$\begin{cases} m\ddot{x} = T(\cos\psi\sin\theta\cos\phi + \sin\psi\sin\phi) \\ m\ddot{y} = T(\sin\psi\sin\theta\cos\phi - \cos\psi\sin\phi) \\ m\ddot{z} = T\cos\theta\cos\phi - mg \\ J_x\dot{\omega}_x + (J_z - J_y)\omega_y\omega_z = \tau_x \\ J_y\dot{\omega}_y + (J_x - J_z)\omega_x\omega_z = \tau_y \\ J_z\dot{\omega}_z + (J_y - J_x)\omega_x\omega_y = \tau_z \end{cases}$$

Drone model (hybrid rep.)



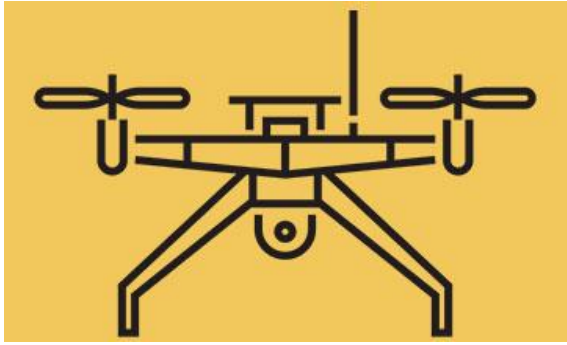
with $$\begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} = \begin{bmatrix} 1 & 0 & -\sin\theta \\ 0 & \cos\phi & \cos\theta\sin\phi \\ 0 & -\sin\phi & \cos\theta\cos\phi \end{bmatrix} \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix}$$ **(angular tranform)**

Nonlinear system, dim(state)=6 * 2nd-order diff. eq = 12, and 4 inputs.

4 Flat variables (or vector F with 4 components)

Let us choose $F_1 = x,\ F_2 = y,\ F_3 = z$ and $F_4 = \psi$

Drones are differentially flat! (2/3)



Let us start with the first input T, the thrust:

Isolate trig. terms in first 3 eq, square and add all terms gives:

$$m^2(\ddot{x}^2 + \ddot{y}^2 + (\ddot{z} + g)^2) = T^2$$

(tedious but easy to check)

So that we have

$$T^2 = m^2(\ddot{F}_1^2 + \ddot{F}_2^2 + (\ddot{F}_3 + g)^2)$$

and finally

$$\boxed{T = m \pm \sqrt{\ddot{F}_1^2 + \ddot{F}_2^2 + (\ddot{F}_3 + g)^2}}$$

(up or down!)

$$\begin{cases} m\ddot{x} = T(\cos\psi\sin\theta\cos\phi + \sin\psi\sin\phi) & (1) \\ m\ddot{y} = T(\sin\psi\sin\theta\cos\phi - \cos\psi\sin\phi) & (2) \\ m\ddot{z} = T\cos\theta\cos\phi - mg & (3) \\ J_x\dot{\omega}_x + (J_z - J_y)\omega_y\omega_z = \tau_x & (4) \\ J_y\dot{\omega}_y + (J_x - J_z)\omega_x\omega_z = \tau_y & (5) \\ J_z\dot{\omega}_z + (J_y - J_x)\omega_x\omega_y = \tau_z & (6) \end{cases}$$

$$F_1 = x,\ F_2 = y,\ F_3 = z \text{ and } F_4 = \psi$$

What about pitch?

Multiply (1) by cos ψ minus multiply (2) by sin ψ and divide by (3)+mg:

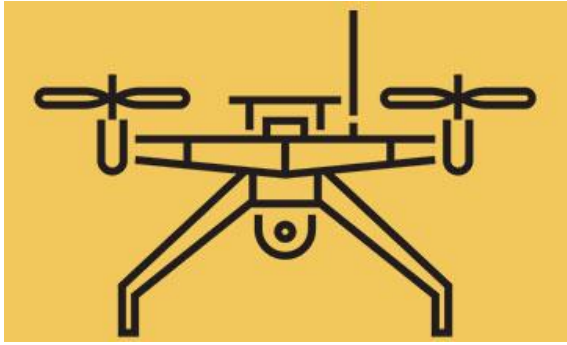
$$m\ddot{x}\cos\psi + m\ddot{y}\sin\psi = T\cos\phi\sin\theta \quad (7)$$

divide by

$$m\ddot{z} + mg = T\cos\phi\cos\theta \quad (8)$$

$$\Longrightarrow \boxed{\theta = \arctan\frac{\ddot{F}_1\cos F_4 + \ddot{F}_2\sin F_4}{\ddot{F}_3 + g}}$$

Drones are differentially flat! (3/3)



What about roll?

- (1) * sin ψ - (2) * cos ψ:

$$m\ddot{x}\sin\psi - m\ddot{y}\cos\psi = T\sin\phi$$

- $\sqrt{(7)^2+(8)^2}$:

$$\sqrt{(m\ddot{z}+mg)^2 + (m\ddot{x}\cos\psi + m\ddot{y}\sin\psi)^2} = T\cos\phi$$

and divide the above, so that we have

$$\boxed{\phi = \arctan\frac{\ddot{F}_1\sin F_4 - \ddot{F}_2\cos F_4}{\sqrt{(\ddot{F}_3+g)^2 + (\ddot{F}_1\cos F_4 + \ddot{F}_2\sin F_4)^2}}}$$

$$\begin{array}{l} (1) \\ (2) \\ (3) \\ (4) \\ (5) \\ (6) \end{array} \left\{ \begin{array}{l} m\ddot{x} = T(\cos\psi\sin\theta\cos\phi + \sin\psi\sin\phi) \\ m\ddot{y} = T(\sin\psi\sin\theta\cos\phi - \cos\psi\sin\phi) \\ m\ddot{z} = T\cos\theta\cos\phi - mg \\ J_x\dot{\omega}_x + (J_z - J_y)\omega_y\omega_z = \tau_x \\ J_y\dot{\omega}_y + (J_x - J_z)\omega_x\omega_z = \tau_y \\ J_z\dot{\omega}_z + (J_y - J_x)\omega_x\omega_y = \tau_z \end{array} \right.$$

$$F_1 = x,\ F_2 = y,\ F_3 = z \text{ and } F_4 = \psi$$

Let us briefly have a look at τ_x (ω's are obtained from ang. transf.):

- Simply isolate τ_x in (4) to get

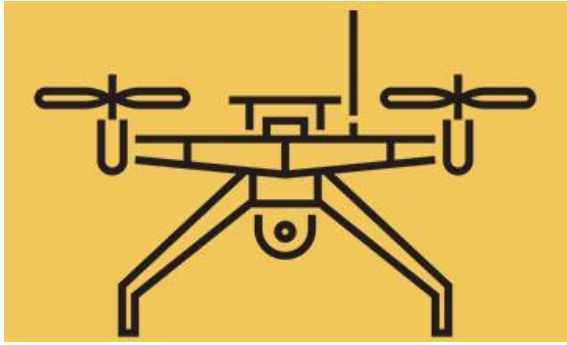
$$\tau_x = (J_z - J_y)\omega_y\omega_z + J_x\dot{\omega}_x \quad \Longrightarrow$$

$$\boxed{\tau_x = (J_z - J_y)\omega_y(\dot{F}, \ddot{F}, \cdots)\omega_z(\dot{F}, \ddot{F}, \cdots) + J_x\dot{\omega}_x(\ddot{F}, \dddot{F}, \cdots)}$$

and similarly for τ_y and τ_z!

SDU

Trivial but important remark



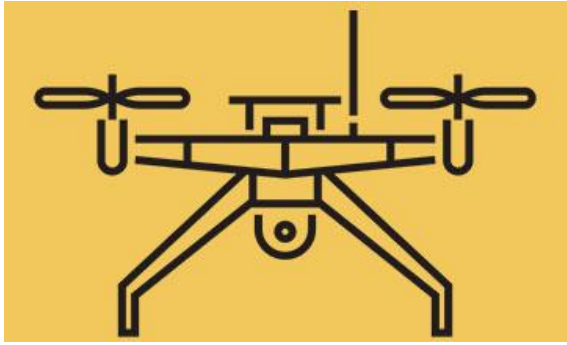
In the above approach, if the feedforward controller does most of the work, feedback control is also (almost) always needed in practise!

(to correct for modelling uncertainties, error dues to noise, small delays, etc.)

→ **Tracking control**

Application: high-speed cornering

Application: drone with payload



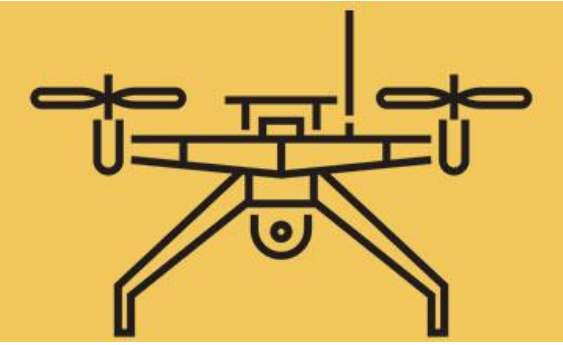
Differential Flatness based Path Planning with Direct Collocation on Hybrid Modes for a Quadrotor with a Cable-Suspended Load

Jun Zeng, Prasanth Kotaru, Mark W. Mueller and Koushil Sreenath

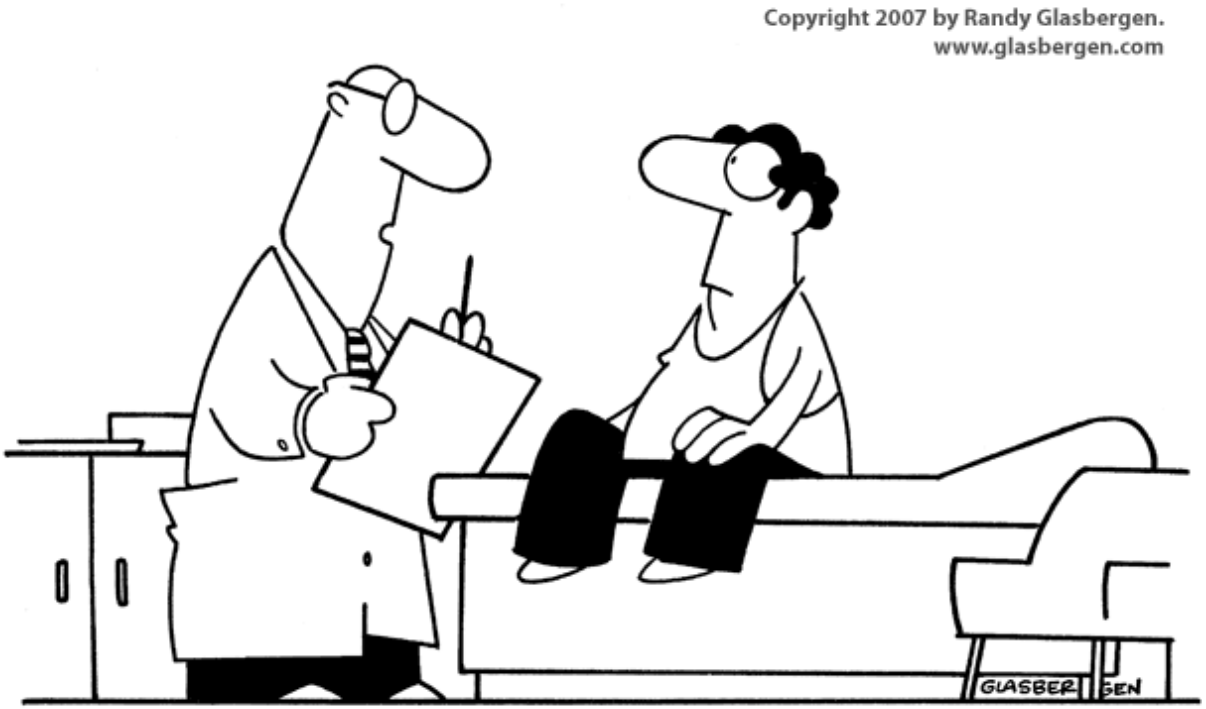
hybrid-robotics.berkeley.edu

Berkeley
UNIVERSITY OF CALIFORNIA

Application: aggressive drone maneuvers



Let us take a break



"Stop drinking coffee for a week, then start drinking it again.
It's the same effect as rebooting your computer."

Feedforward for a Roomba (1/2)

- Q1) In a Matlab script, make a program that calculate the coefficients of 2 3rd order polynomials using boundary conditions x(0)=(0,0,pi/4) and x(T)=(1,5,3pi/4) with T=10 seconds, with initial and final longitudinal velocities at 1m/s (ie $u_1(0)=u_1(T)=1$).
- Q2) Download the Simulink template 'flatness_temp.slx' from its learning and check that it works.
- Q3) In the Feedforward controller subsystem, use a Matlab function block that will take the alpha coefficients of a 3rd order polynomial (along both dimensions x_1 and x_2) and time as inputs, and calculate the vector of flat outputs (F and its derivatives)

Feedforward for a Roomba (2/2)

- Q4) In the Feedforward controller subsystem, use another Matlab function block that will take the flat output vector as inputs and calculate the feedforward control signal $u_d(t)$.
- Q5) Apply your controller to the model and check that it works. Modify it so that the robot stays on x(T) afterwards.
- Q6) Modify your programs so that the robot goes to other positions.

Use the Symbolic toolbox for feedforward

(we would like to avoid computing the feedforward expression of the controller by hand and instead use the symbolic toolbox)

- Q7) In a Matlab script, use the command 'syms' to declare time (as t) all variables (<u>and their derivatives</u>) of the model (eg 'x1(t)').
- Q8) Use the help and the '==' operator to describe the mathematical model of the Roomba.
- Q9) Manipulate (by programming) these equations to obtain the 2 mappings 'mu' of flatness theory (you can use the commands 'isolate', 'subs' and more to do that).